



Instituto Politécnico
de Viana do Castelo

LICENCIATURA EM ENGENHARIA INFORMÁTICA

OPÇÃO II – APRENDIZAGEM ORGANIZACIONAL / SISTEMAS DE SUPORTE À DECISÃO

Deepfake Detector

N.º 29109 – João Salgado

joaojosesalgado@ipvc.pt

Nº31391 – Pedro Morais

p.morais@ipvc.pt

Jorge Ribeiro

• jribeiro@estg.ipvc.pt

Abel Dantas

• adantas@estg.ipvc.pt

■ Introdução

- O que são Deepfakes?

Deepfakes são imagens, vídeos ou áudios manipulados, onde o rosto ou a voz de uma pessoa é substituído pelo de outra de forma realista. O termo vem de *deep learning* (aprendizagem profunda) + *fake* (falso).

- Porque é um problema?

Com o rápido avanço das ferramentas, criar um deepfake convincente está cada vez mais acessível, qualquer pessoa com um computador consegue fazê-lo em minutos. Isto cria riscos sérios:

- **Desinformação** : vídeos falsos de figuras públicas a dizer coisas que nunca disseram;
- **Fraude** : usar o rosto de outra pessoa para enganar sistemas de autenticação;
- **Manipulação** : conteúdo fabricado para influenciar opiniões políticas ou sociais.

- A nossa resposta

Este projeto desenvolve uma aplicação web de deteção automática de deepfakes, utilizando Deep Learning para analisar imagens faciais e classificá-las como **REAL** ou **FAKE**, com o grau de confiança.

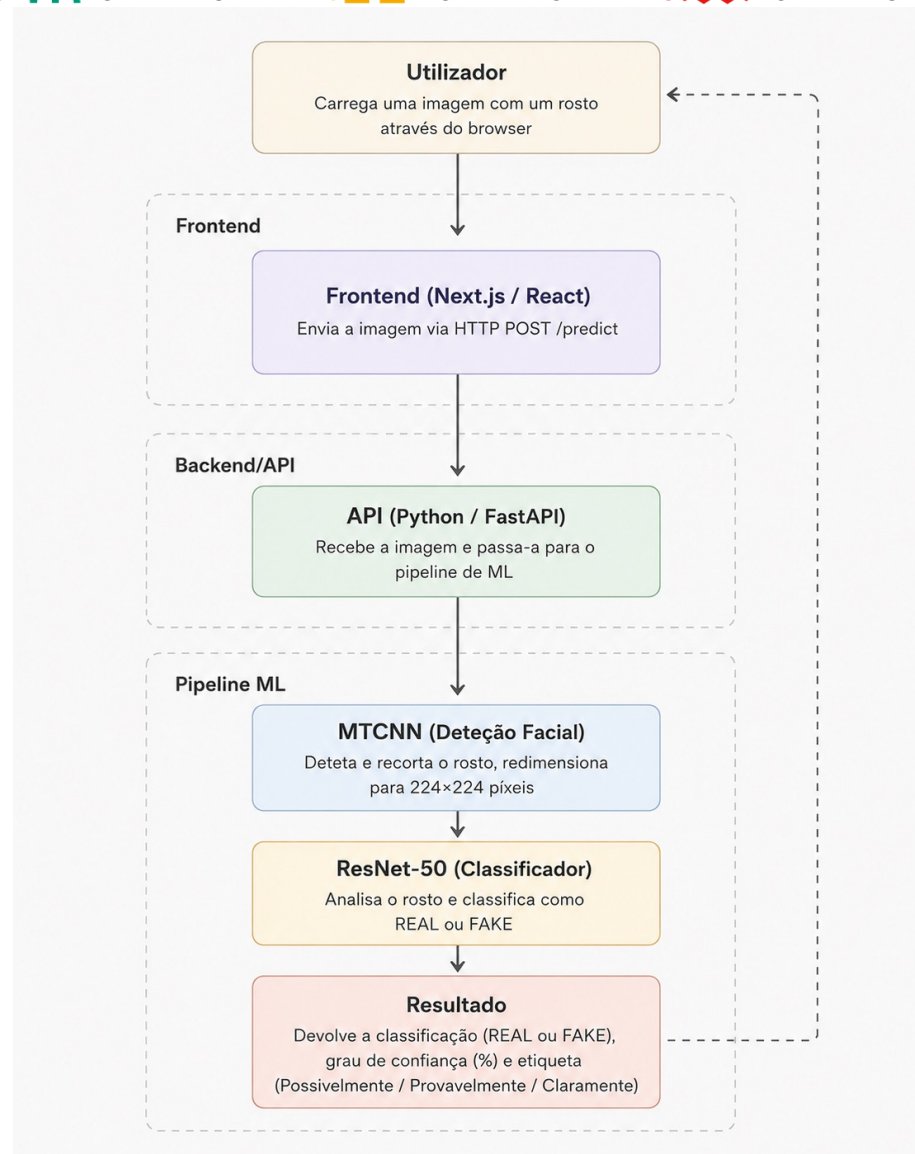
■ Objetivos

- Desenvolver um modelo de Deep Learning capaz de classificar imagens faciais como **REAL** ou **FAKE**;
- Criar uma aplicação web acessível onde qualquer utilizador possa carregar uma imagem e obter um resultado imediato;
- Apresentar o resultado com grau de confiança, distinguindo entre possivelmente, provavelmente e claramente falso/real;
- Compreender as limitações dos modelos de deteção e o que influencia o seu desempenho.

■ Datasets

Dataset	Criador	Tipo	Imagens	Descrição
CelebDF	Johns Hopkins University	Face Swap	65 381	Deepfakes de alta qualidade de celebridades
FaceForensics++ (FF++)	Technical University of Munich	Face Swap	20 351 (todas FAKE)	Manipulações faciais com 4 métodos: DeepFakes, Face2Face, FaceSwap e NeuralTextures
DFDC	Meta (Facebook)	Face Swap	7 732	Dataset da competição Deepfake Detection Challenge, com grande diversidade de atores e condições
140K Real and Fake Faces	Bojan Tunguz (Kaggle)	GAN	60 000	70K faces reais do Flickr e 70K faces geradas por GAN, equilibrado entre real e falso
StyleGAN3	NVIDIA	GAN	20 000	10K faces sintéticas geradas pelo StyleGAN3, um dos geradores de faces mais realistas existentes
FFHQ	NVIDIA	Real	20 000 (todas REAL)	Flickr-Faces-HQ, 70K faces reais de alta qualidade, usadas como referência no treino

Arquitetura do Projeto



■ Arquitetura do Projeto

1. Utilizador

- Acede à aplicação pelo browser e carrega uma imagem com um rosto para ser analisada.

2. Frontend (Next.js / React)

- Interface web que recebe a imagem e envia-a para a API através de um pedido HTTP POST para /predict.

3. API (Python / FastAPI)

- Recebe a imagem enviada pelo frontend e coordena todo o pipeline de deteção, passa a imagem ao MTCNN e aguarda o resultado do classificador.

4. MTCNN - Deteção Facial

- Localiza automaticamente o rosto na imagem, recorta apenas essa região e redimensiona para 224×224 píxeis. Garante que o classificador analisa apenas o rosto e não elementos irrelevantes como o fundo ou a roupa.

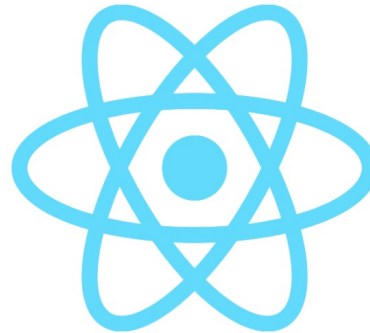
5. ResNet-50 - Classificador

- Rede neuronal profunda com 50 camadas que analisa o rosto em busca de padrões e artefactos invisíveis ao olho humano, inconsistências na textura da pele, irregularidades nos contornos e padrões anómalos nos olhos ou cabelo.

6. Resultado

- Devolve ao utilizador a classificação (**REAL** ou **FAKE**), o grau de confiança em percentagem e a etiqueta correspondente, *Possivelmente*, *Provavelmente* ou *Claramente*.

■ Ferramentas e Tecnologias



kaggle



PyTorch



FastAPI

■ Ferramentas e Tecnologias

Camada	Tecnologia	Função
Frontend	Next.js + React	Interface web para carregar imagens e ver resultados
API	Python + FastAPI	Recebe a imagem e coordena o pipeline de deteção
Servidor API	Uvicorn	Permite à API processar vários pedidos em simultâneo
Modelo	PyTorch	Framework base para construir e correr redes neuronais
Classificador	ResNet-50	Rede neuronal que classifica a face como REAL ou FAKE
Deteção Facial	MTCNN (facenet-pytorch)	Deteta e recorta automaticamente o rosto da imagem
Processamento de Imagem	Pillow + NumPy	Convertem e preparam a imagem para o modelo
Treino	Kaggle (GPU T4)	Plataforma onde o modelo foi treinado
Linguagens	Python / JavaScript	Linguagens base do projeto

Treino

1. Como treinámos o modelo?

- Partimos de um modelo já existente (ResNet-50) que já sabia reconhecer imagens do mundo real, e ensinámo-lo a distinguir faces reais de faces falsas. A este processo chama-se *fine-tuning*.

2. Para o modelo aprender melhor, fizemos duas coisas:

- **Data Augmentation** : durante o treino, aplicámos pequenas alterações aleatórias às imagens (rotações, variações de brilho, desfoque, recortes) para que o modelo aprendesse a lidar com imagens imperfeitas e não apenas com casos ideais
- **Balanceamento** : como tínhamos mais imagens FAKE do que REAL, usámos uma técnica que garante que o modelo vê ambas as classes de forma equilibrada durante o treino.

3. Quanto tempo demorou?

- O modelo foi treinado durante **8 épocas**, ou seja, viu o conjunto completo de ~193K imagens 8 vezes. Cada passagem completa permitiu ao modelo ajustar o que tinha aprendido.

4. Resultados:

- O modelo melhorou consistentemente ao longo do treino. O melhor resultado foi atingido na **época 7**:

96.4% de imagens classificadas corretamente

AUC de 0.9950 - muito próximo do valor máximo (1.0)

F1-Score de 0.9692 - bom equilíbrio entre detetar fakes e evitar falsos alarmes

Treino

— Época 1/8 —
Train: loss=0.3905, acc=0.8178
Val: loss=0.2547, acc=0.8923, AUC=0.9591, F1=0.9085
Novo melhor modelo (AUC=0.9591)

— Época 2/8 —
Train: loss=0.2495, acc=0.8915
Val: loss=0.2098, acc=0.9100, AUC=0.9795, F1=0.9208
Novo melhor modelo (AUC=0.9795)

— Época 3/8 —
Train: loss=0.2009, acc=0.9138
Val: loss=0.1408, acc=0.9446, AUC=0.9885, F1=0.9527
Novo melhor modelo (AUC=0.9885)

— Época 4/8 —
Train: loss=0.1687, acc=0.9283
Val: loss=0.1173, acc=0.9541, AUC=0.9914, F1=0.9611
Novo melhor modelo (AUC=0.9914)

— Época 5/8 —
Train: loss=0.1524, acc=0.9364
Val: loss=0.1132, acc=0.9553, AUC=0.9936, F1=0.9615
Novo melhor modelo (AUC=0.9936)

— Época 6/8 —
Train: loss=0.1386, acc=0.9420
Val: loss=0.1084, acc=0.9594, AUC=0.9943, F1=0.9652
Novo melhor modelo (AUC=0.9943)

— Época 7/8 —
Train: loss=0.1298, acc=0.9457
Val: loss=0.0950, acc=0.9640, AUC=0.9950, F1=0.9692
Novo melhor modelo (AUC=0.9950)

— Época 8/8 —
Train: loss=0.1276, acc=0.9473
Val: loss=0.1114, acc=0.9576, AUC=0.9942, F1=0.9636
Sem melhoria (1/4)

Treino completo! Melhor AUC val: 0.9950

■ Limitações e Dificuldades

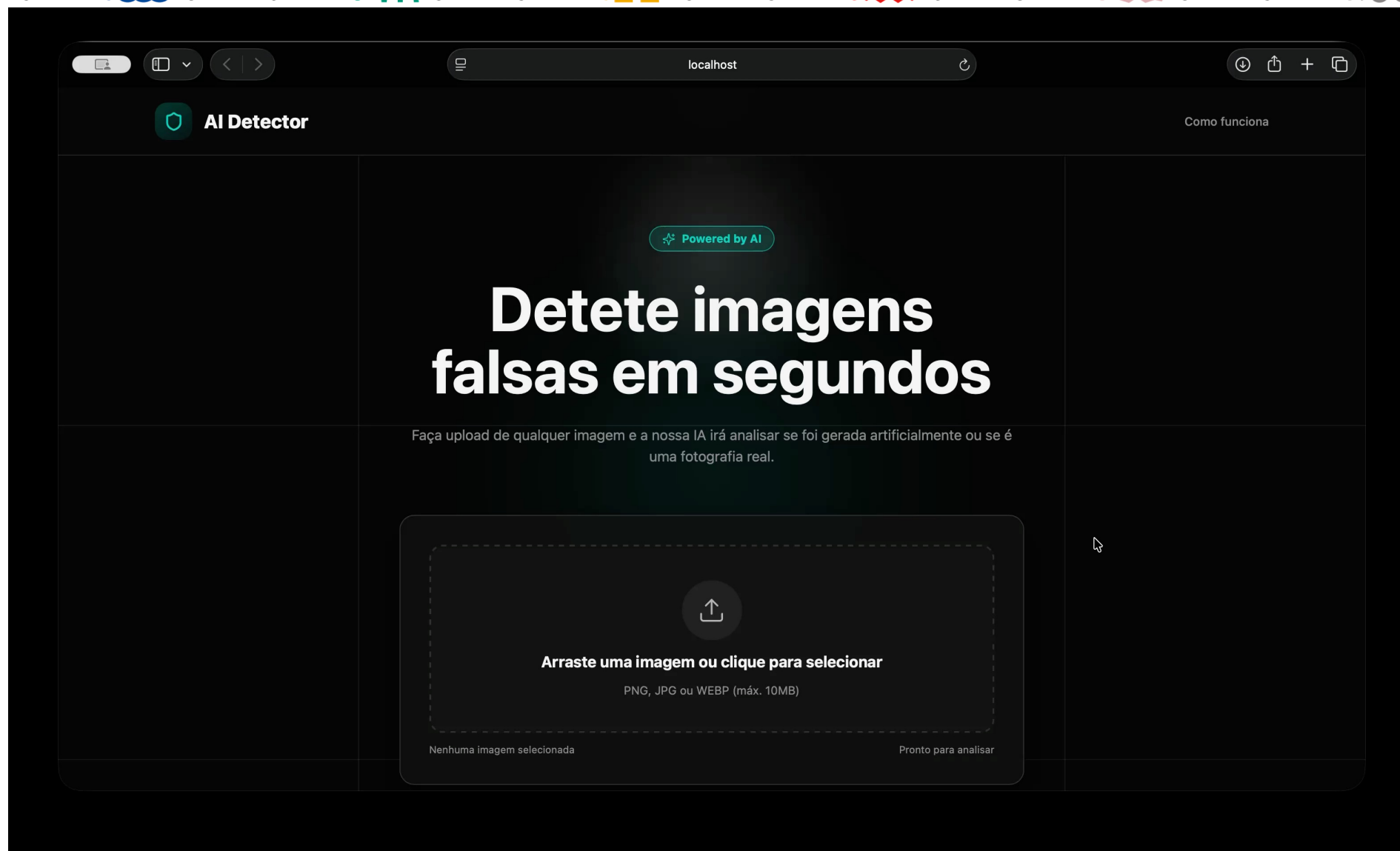
Limitações do modelo:

- O modelo só consegue detetar deepfakes semelhantes aos que viu durante o treino, técnicas mais recentes ou diferentes das usadas nos 6 datasets podem não ser detetadas
- Quanto mais realista e recente for a técnica de geração, maior a probabilidade de o modelo falhar
- O modelo analisa apenas imagens estáticas, não consegue analisar vídeos diretamente
- Não é infalível, existe sempre uma margem de erro que pode levar a falsos positivos ou falsos negativos

Dificuldades encontradas:

- Encontrar o melhor conjunto de datasets de fontes diferentes com formatos, qualidades e resoluções distintas
- Tentámos adicionar mais datasets para melhorar a capacidade de deteção, mas com muitos datasets de tipos diferentes e poucos exemplos de cada tipo, o modelo ficava confuso e o desempenho piorava
- O treino com GPU gratuita do Kaggle tinha limitações de tempo, adicionar mais dados ou aumentar o número de épocas tornava o treino demasiado longo
- O número de épocas de treino teve de ser ajustado com cuidado, poucas épocas resultavam num modelo pouco preciso, mas mais épocas aumentavam significativamente o tempo de treino

Demonstração



■ Conclusão

O que alcançámos:

- Desenvolvemos uma aplicação web capaz de analisar imagens faciais e classificá-las como REAL ou FAKE com grau de confiança
- Treinamos um modelo com ~193K imagens de 6 datasets distintos, cobrindo as principais técnicas de geração e manipulação de faces
- O pipeline completo, desde a deteção facial com MTCNN até à classificação com ResNet-50, funciona em tempo real

O que aprendemos:

- A qualidade e diversidade dos dados de treino é tão importante quanto a arquitetura do modelo
- Mais dados nem sempre significa melhor desempenho, a distribuição e equilíbrio entre classes é fundamental
- Detetar deepfakes é um problema em constante evolução, à medida que as técnicas de geração melhoram, os modelos de deteção têm de acompanhar

Trabalho futuro:

- Expandir o modelo para suportar análise de vídeos
- Melhorar o desempenho com mais datasets diferentes, tendo mais épocas de treino e maior capacidade de GPU

■ Referências

- *Celeb-DF*:
<https://www.kaggle.com/datasets/amanrawat001/celeb-df-preprocessed>
- *FaceForensics++*:
<https://www.kaggle.com/datasets/greatgamedota/faceforensics>
- *The Deep Fake Detection Challenge Dataset* .MetaAI :
<https://www.kaggle.com/datasets/pedromorais05/dfdc-faces>
- *140K Real and Fake Faces*:
<https://www.kaggle.com/datasets/xhlulu/140k-real-and-fake-faces>
- *Alias-Free Generative Adversarial Networks (StyleGAN3)*. NVIDIA :
<https://www.kaggle.com/datasets/troykueh/real-vs-fake-faces-stylegan3>
- *A Style-Based Generator Architecture for GANs (FFHQ)*. NVIDIA :
<https://www.kaggle.com/datasets/greatgamedota/ffhq-face-data-set>

o teu • de partida



ipvc

Instituto Politécnico
de Viana do Castelo

www.ipvc.pt